Artificial Jewellery

E-Commerce Platform

Detailed Architecture & Design Document

Enhanced Edition with Visual Diagrams

Generated: December 26, 2025

Version 2.0 - Enhanced

Table of Contents

1. Introduction 3

2. Tech Stack 4

3. High-Level Architecture 5

4. NestJS Backend Modules 6

5. Sanity CMS Content Schema 7

6. Database Schema (Prisma) 8

7. Database Relationships 9

8. Product Flow & API Endpoints 10

9. Ordering Logic 11

10. Discount Logic 12

11. Payment Integration (Razorpay) 13

12. Payment Flow Diagram 14

13. AI Recommendation Readiness 15

14. Future Extensions 16

15. Example Scenarios 17

16. Conclusion 18

1. Introduction

This document provides a comprehensive architecture and design specification for an Artificial Jewellery E-Commerce Platform. The platform is designed to be highly scalable, maintainable, and ready for future enhancements including AI-powered jewellery recommendations based on dress images. The architecture leverages modern technologies including NestJS for the backend, Next.js for the frontend, Sanity CMS for content management, PostgreSQL with Prisma ORM for data persistence, and Razorpay for payment processing. The system is designed with modularity and scalability as core principles, ensuring it can grow with business needs. **Key Design Principles:**

• Modular architecture for independent scaling
• Separation of content (CMS) and transactional data (Database)
• Type-safe development with Prisma ORM
• RESTful API design for frontend integration
• Future-ready for AI/ML integration

2. Tech Stack

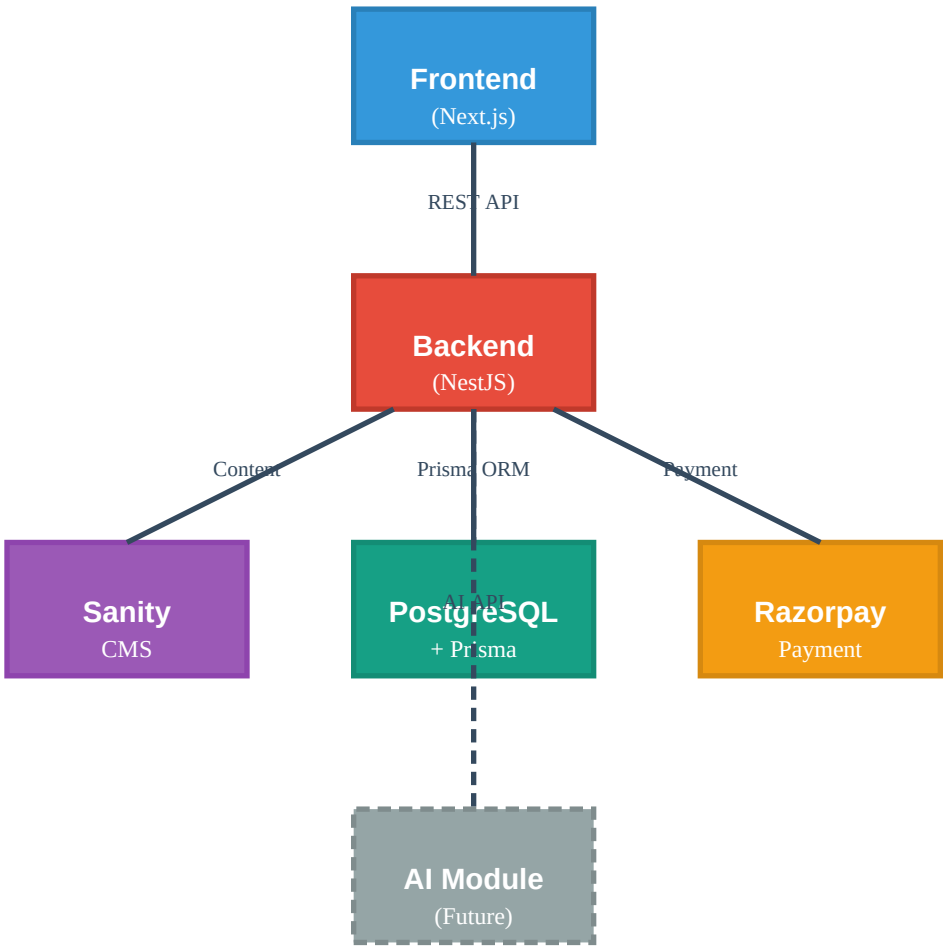
Component	Technology	Purpose
Frontend Framework	Next.js 14+	Server-side rendering, React-based UI
Backend Framework	NestJS	Modular, scalable Node.js backend
Content Management	Sanity CMS	Headless CMS for product content
Database	PostgreSQL	Relational database for transactional data
ORM	Prisma	Type-safe database access
Payment Gateway	Razorpay	Payment processing
Future AI Module	Python/ML (TBD)	Image-based jewellery recommendations

3. High-Level Architecture

3.1 System Overview

The system follows a layered architecture pattern with clear separation of concerns: **Presentation Layer (Next.js Frontend):** Handles user interactions, displays product catalogs, manages shopping carts, and processes checkout flows. Implements server-side rendering for SEO and performance. **Application Layer (NestJS Backend):** Contains business logic, API endpoints, authentication, order processing, and integration with external services. Organized into feature modules for maintainability. **Data Layer:** Consists of PostgreSQL database (transactional data) and Sanity CMS (content data). Prisma ORM provides type-safe access to PostgreSQL. **External Services:** Razorpay for payments, Sanity CMS API for content, and future AI service for recommendations.

3.2 Architecture Diagram



Key Interactions:
• Frontend communicates with Backend via REST API
• Backend fetches product content from Sanity CMS

• Backend stores transactional data in PostgreSQL via Prisma ORM
• Backend processes payments through Razorpay
• Backend will integrate with AI module for recommendations (future - shown in dashed lines)

4. NestJS Backend Modules

The NestJS backend is organized into feature modules following Domain-Driven Design principles. Each module is self-contained with its own controllers, services, and DTOs.

Module	Responsibility	Key Endpoints
Catalog	Product management, listings, search	GET /products, GET /products/:id, GET /categories
Cart	Shopping cart management	POST /cart, GET /cart, PUT /cart/:id, DELETE /cart/:id
Order	Order creation and management	POST /orders, GET /orders/:id, GET /orders
Payment	Payment processing via Razorpay	POST /payments/create, POST /payments/verify
User	User authentication and profiles	POST /auth/register, POST /auth/login, GET /user/profile
Admin	Administrative operations	POST /admin/products, PUT /admin/products/:id

4.1 Catalog Module Structure

```
Folder Structure:
src/catalog/
■■■ catalog.module.ts
■■■ catalog.controller.ts
■■■ catalog.service.ts
■■■ catalog.repository.ts
■■■ sanity-adapter.service.ts
■■■ dto/
■ ■■■ product-response.dto.ts
■ ■■■ variant-response.dto.ts
■ ■■■ product-query.dto.ts
■■■ entities/
■■■ product.entity.ts
```

4.2 Example Catalog Service Code

```
// catalog.service.ts import { Injectable } from '@nestjs/common'; import {
PrismaService } from '../prisma/prisma.service'; import {
SanityAdapterService } from './sanity-adapter.service'; @Injectable() export
class CatalogService { constructor( private prisma: PrismaService, private
sanityAdapter: SanityAdapterService, ) {} async getProducts(query:
ProductQueryDto) { // Query database const products = await
this.prisma.product.findMany({ where: { isActive: true, stock: { gt: 0 } },
include: { variants: true }, orderBy: [ { stock: 'desc' }, { displayRank:
'asc' }, { createdAt: 'desc' } ], take: query.limit, skip: (query.page - 1) *
query.limit, }); // Fetch Sanity content for each product const
productsWithContent = await Promise.all( products.map(async (product) => {
const sanityData = await this.sanityAdapter .getProduct(product.sanityId);
return this.mergeProductData(product, sanityData); }) ); return
productsWithContent; } private mergeProductData(dbProduct, sanityData) {
return { ...dbProduct, title: sanityData.title, description:
sanityData.description, images: sanityData.images, badge: sanityData.badge,
tags: sanityData.tags, }; } }
```

5. Sanity CMS Content Schema

Sanity CMS stores product content including descriptions, images, tags, and metadata. The backend merges this content with database records that contain business logic data (stock, prices, variants).

5.1 Product Schema

```
// schemas/product.js export default { name: 'product', title: 'Product', type: 'document', fields: [ { name: 'sanityId', title: 'Sanity ID', type: 'string', description: 'Unique identifier matching DB product.sanityId' }, { name: 'title', title: 'Product Title', type: 'string', validation: Rule => Rule.required() }, { name: 'description', title: 'Description', type: 'text', validation: Rule => Rule.required() }, { name: 'images', title: 'Product Images', type: 'array', of: [{ type: 'image', options: { hotspot: true } }] }, { name: 'category', title: 'Category', type: 'reference', to: [{ type: 'category' }] }, { name: 'tags', title: 'Tags', type: 'array', of: [{ type: 'string' }], options: { layout: 'tags' } }, { name: 'badge', title: 'Badge (e.g., "New", "Sale")', type: 'string', options: { list: ['New', 'Sale', 'Trending', 'Featured'] } }, { name: 'displayRank', title: 'Display Rank (lower = higher priority)', type: 'number', description: 'Products with lower numbers appear first' } ], preview: { select: { title: 'title', media: 'images.0' } } }
```

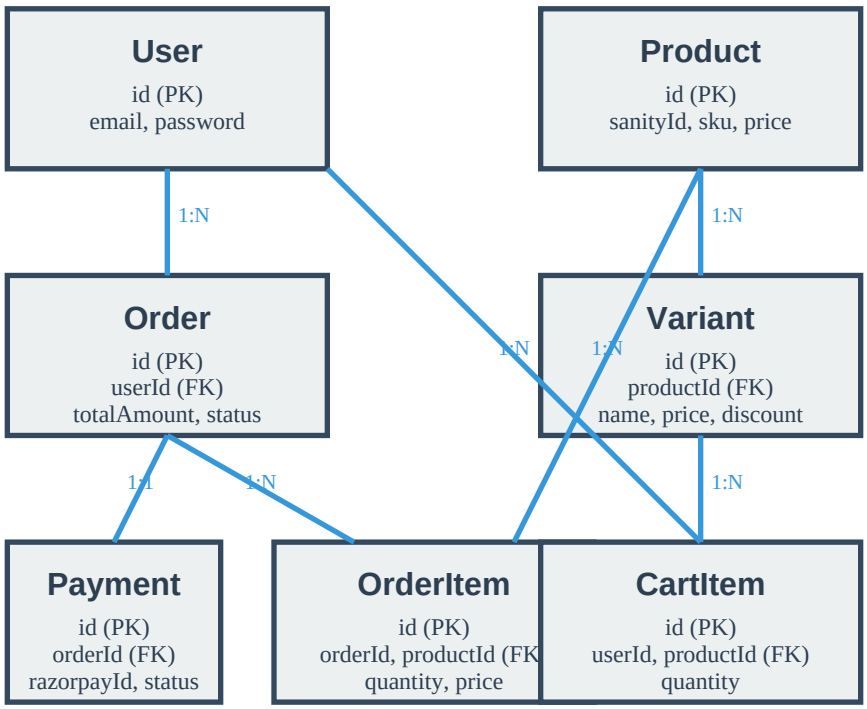
6. Database Schema (Prisma)

The database schema is defined using Prisma ORM, providing type-safe database access. The schema includes models for users, products, variants, orders, payments, and shopping carts.

```
// prisma/schema.prisma generator client { provider = "prisma-client-js" }
datasource db { provider = "postgresql" url = env("DATABASE_URL") } model
User { id String @id @default(cuid()) email String @unique password String
name String? phone String? createdAt DateTime @default(now()) updatedAt
DateTime @updatedAt orders Order[] cartItems CartItem[] @@map("users") }
model Product { id String @id @default(cuid()) sanityId String @unique sku
String @unique basePrice Decimal @db.Decimal(10, 2) stock Int @default(0)
isActive Boolean @default(true) createdAt DateTime @default(now()) updatedAt
DateTime @updatedAt variants Variant[] orderItems OrderItem[] cartItems
CartItem[] @@index([sanityId]) @@index([isActive, stock]) @@map("products") }
model Variant { id String @id @default(cuid()) productId String name String
// e.g., "Gold", "Silver" price Decimal? @db.Decimal(10, 2) // Overrides
product.basePrice if set stock Int @default(0) sku String @unique discount
Decimal? @db.Decimal(5, 2) // Percentage discount (variant-level) isActive
Boolean @default(true) product Product @relation(fields: [productId],
references: [id], onDelete: Cascade) @@index([productId]) @@map("variants") }
model Category { id String @id @default(cuid()) name String @unique slug
String @unique description String? parentId String? parent Category?
@relation("CategoryHierarchy", fields: [parentId], references: [id]) children
Category[] @relation("CategoryHierarchy") @@index([slug]) @@map("categories")
} model CartItem { id String @id @default(cuid()) userId String productId
String variantId String? quantity Int @default(1) createdAt DateTime
@default(now()) user User @relation(fields: [userId], references: [id],
onDelete: Cascade) product Product @relation(fields: [productId], references:
[id], onDelete: Cascade) @@unique([userId, productId, variantId])
@@map("cart_items") } model Order { id String @id @default(cuid()) userId
String status OrderStatus @default(PENDING) totalAmount Decimal
@db.Decimal(10, 2) subtotal Decimal @db.Decimal(10, 2) discount Decimal
@db.Decimal(10, 2) @default(0) shippingCost Decimal @db.Decimal(10, 2)
@default(0) createdAt DateTime @default(now()) updatedAt DateTime @updatedAt
user User @relation(fields: [userId], references: [id]) orderItems
OrderItem[] payment Payment? shippingInfo ShippingInfo? @@index([userId,
status]) @@map("orders") } model OrderItem { id String @id @default(cuid())
orderId String productId String variantId String? quantity Int price Decimal
@db.Decimal(10, 2) // Price at time of order discount Decimal @db.Decimal(5,
2) // Applied discount percentage order Order @relation(fields: [orderId],
references: [id], onDelete: Cascade) product Product @relation(fields:
[productId], references: [id]) @@index([orderId]) @@map("order_items") }
model Payment { id String @id @default(cuid()) orderId String @unique
razorpayId String? @unique amount Decimal @db.Decimal(10, 2) status
PaymentStatus @default(PENDING) method String? createdAt DateTime
@default(now()) updatedAt DateTime @updatedAt order Order @relation(fields:
[orderId], references: [id]) @@index([razorpayId]) @@map("payments") } model
ShippingInfo { id String @id @default(cuid()) orderId String @unique name
String address String city String state String pincode String phone String
order Order @relation(fields: [orderId], references: [id])
@@map("shipping_info") } enum OrderStatus { PENDING CONFIRMED PROCESSING
SHIPPED DELIVERED CANCELLED } enum PaymentStatus { PENDING SUCCESS FAILED
REFUNDED }
```

7. Database Relationships

The database schema implements a relational model with clear foreign key relationships. The diagram below illustrates the entity relationships.

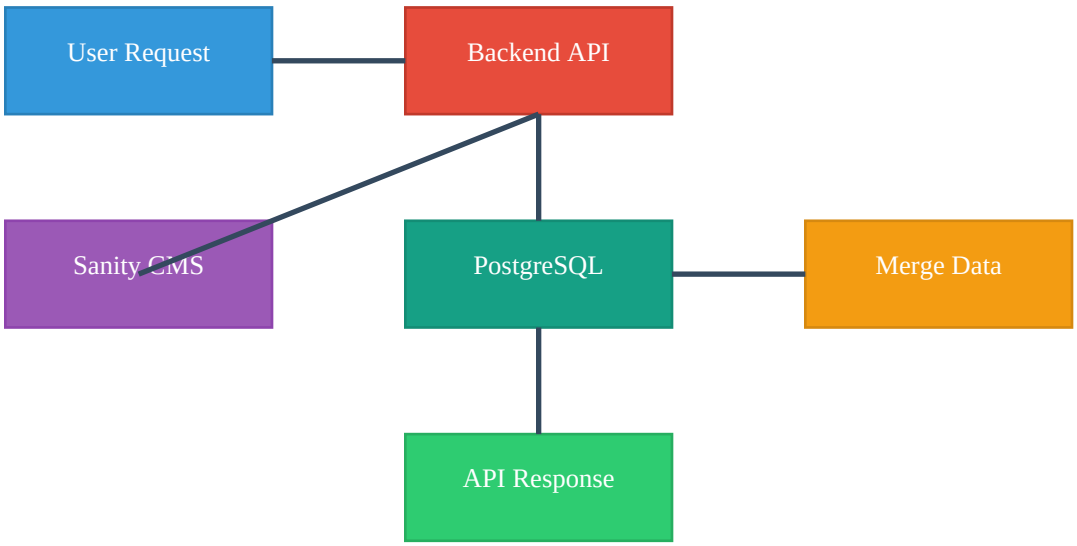


Key Relationships:
• User → Order (1:N): One user can have many orders
• User → CartItem (1:N): One user can have many cart items
• Product → Variant (1:N): One product can have many variants (colors, sizes)
• Product → OrderItem (1:N): One product can appear in many order items
• Product → CartItem (1:N): One product can be in many user carts
• Order → OrderItem (1:N): One order contains many order items
• Order → Payment (1:1): One order has one payment record
• Order → ShippingInfo (1:1): One order has one shipping address

8. Product Flow & API Endpoints

Product Listing Flow:
1. Frontend requests product list: GET /api/catalog/products
2. Backend queries PostgreSQL for active products with stock > 0
3. For each product, fetch content from Sanity CMS using sanityId
4. Merge database data (price, stock) with Sanity content (images, description)
5. Apply ordering logic (in-stock first, then displayRank, then createdAt)
6. Return paginated results

8.1 Product Data Flow



8.2 API Response Example

```
// GET /api/catalog/products?page=1&limit;=20&category;=necklaces { "data": [ { "id": "prod_123", "sanityId": "sanity_prod_123", "title": "Elegant Gold Necklace", "description": "Beautiful artificial gold necklace perfect for parties...", "images": [ "https://cdn.sanity.io/images/abc123/image1.jpg", "https://cdn.sanity.io/images/abc123/image2.jpg" ], "basePrice": 1299.00, "stock": 45, "category": { "id": "cat_1", "name": "Necklaces", "slug": "necklaces" }, "variants": [ { "id": "var_1", "name": "Gold", "price": 1299.00, "stock": 30, "discount": null }, { "id": "var_2", "name": "Rose Gold", "price": 1399.00, "stock": 15, "discount": 10.0 } ], "badge": "New", "displayRank": 1, "tags": ["trending", "elegant", "party"] } ], "pagination": { "page": 1, "limit": 20, "total": 150, "totalPages": 8 } }
```

9. Ordering Logic

Products are ordered using a multi-level sorting strategy to ensure the best user experience and business goals:

```
Sorting Priority (in order):
1. Stock Availability: Products with stock > 0 appear first
2. Display Rank: Lower displayRank values appear first (1, 2, 3...)
3. Newness: Most recently created products appear first Implementation
(TypeScript/JavaScript):
products.sort((a, b) => {
// Priority 1: Stock availability
if (a.stock > 0 && b.stock === 0) return -1;
if (a.stock === 0 && b.stock > 0) return 1;

// Priority 2: Display rank (if set)
if (a.displayRank !== null && b.displayRank !== null) {
return a.displayRank - b.displayRank;
}
if (a.displayRank !== null) return -1;
if (b.displayRank !== null) return 1;

// Priority 3: Newness
return new Date(b.createdAt) - new Date(a.createdAt);
}); Database Query (Prisma):
const products = await prisma.product.findMany({
where: {
isActive: true,
stock: { gt: 0 } // Optional: filter out-of-stock
},
include: { variants: true },
orderBy: [
{ stock: 'desc' }, // In-stock first
{ displayRank: 'asc' }, // Lower rank first
{ createdAt: 'desc' } // Newest first
],
take: limit,
skip: (page - 1) * limit
}); Example Result:
1. Product A: stock=50, displayRank=1, createdAt=2024-01-15
2. Product B: stock=30, displayRank=2, createdAt=2024-01-10
3. Product C: stock=20, displayRank=null, createdAt=2024-01-20
4. Product D: stock=0, displayRank=1, createdAt=2024-01-05 (out of stock,
appears last)
```

10. Discount Logic

Discounts are applied hierarchically with clear precedence rules to ensure predictable pricing:

```
Discount Hierarchy:
1. Variant-level discount (highest priority) - Overrides product-level
2. Product-level discount - Applied if no variant discount exists
3. No discount - Full price if no discounts set Rules:
• Only one discount can be active at a time
• Discounts are stored as percentages (e.g., 10.0 = 10%)
• Final price calculation: price * (1 - discount/100)
• Discounts are applied at cart calculation time
• Order items store the discounted price for historical accuracy Price
Calculation Implementation:
function calculatePrice(product, variant = null) {
let basePrice = product.basePrice;
let discount = null;

if (variant) {
// Use variant price if set, otherwise use product base price
basePrice = variant.price || product.basePrice;
// Variant discount takes priority
discount = variant.discount || product.discount || null;
} else {
discount = product.discount || null;
}

if (discount && discount > 0) {
return basePrice * (1 - discount / 100);
}
return basePrice;
} Example Scenarios:
Scenario 1: Product base price = ■1,299, no discounts
→ Final price: ■1,299
Scenario 2: Product base price = ■1,299, product discount = 15%
→ Final price: ■1,299 * 0.85 = ■1,104.15
Scenario 3: Variant "Rose Gold" price = ■1,399, variant discount = 10%
→ Final price: ■1,399 * 0.90 = ■1,259.10
Scenario 4: Product discount = 15%, Variant discount = 10% (variant wins)
→ Final price: Uses variant discount (10%)
```

11. Payment Integration (Razorpay)

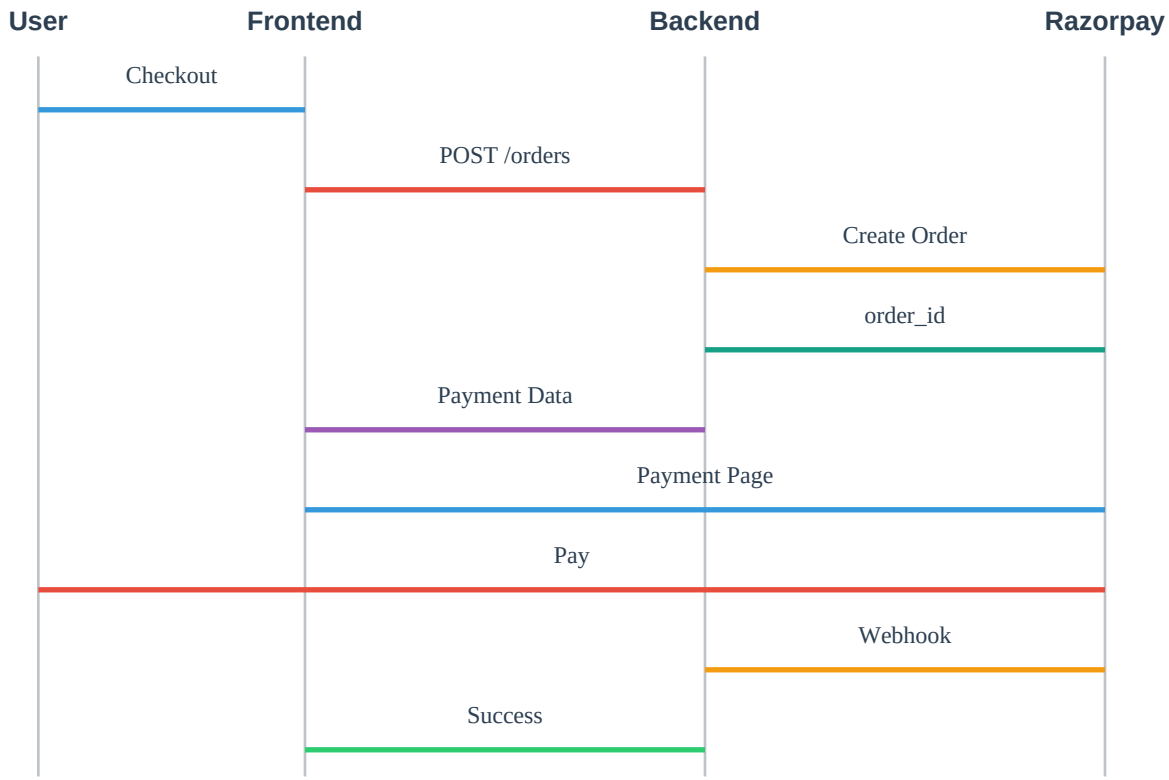
Razorpay integration handles payment processing with webhook support for payment status updates. The integration follows Razorpay's best practices for security and reliability.

Payment Flow Steps:
1. User initiates checkout: POST /api/orders
2. Backend creates order with status PENDING
3. Backend creates Razorpay order: POST /api/payments/create
4. Razorpay returns order_id and key
5. Frontend redirects to Razorpay payment page
6. User completes payment on Razorpay
7. Razorpay webhook: POST /api/payments/webhook
8. Backend verifies payment signature
9. Backend updates order status to CONFIRMED
10. Backend updates payment record with razorpayId and status **Webhook Security:**
• Verify Razorpay signature using secret key
• Validate order amount matches
• Idempotent processing (check if already processed)
• Return 200 OK immediately, process asynchronously **Example Payment Service Code:**

```
@Injectable()
export class PaymentService {
async createRazorpayOrder(orderId: string, amount: number) {
const razorpayOrder = await this.razorpay.orders.create({
amount: amount * 100, // Convert to paise
currency: 'INR',
receipt: orderId,
});

await this.prisma.payment.update({
where: { orderId },
data: { razorpayId: razorpayOrder.id }
});

return {
orderId: razorpayOrder.id,
key: process.env.RAZORPAY_KEY_ID,
amount: razorpayOrder.amount
};
}

async verifyWebhook(signature: string, orderId: string, paymentId: string) {
const payload = orderId + '|' + paymentId;
const expectedSignature = crypto
.createHmac('sha256', process.env.RAZORPAY_SECRET)
.update(payload)
.digest('hex');

return signature === expectedSignature;
}
}
```

12. Payment Flow Diagram

The following sequence diagram illustrates the complete payment flow from checkout initiation to payment confirmation:



13. AI Recommendation Readiness

The architecture is designed to accommodate future AI integration for image-based jewellery recommendations. The system will accept dress images and suggest matching jewellery pieces based on color, style, and occasion analysis.

Planned AI Module Architecture:
1. **Image Upload Endpoint:** POST /api/ai/recommend
• Accepts image file (dress photo)
• Validates image format (JPEG, PNG) and size (< 10MB)
• Stores temporarily or sends directly to AI service **AI Service Integration:**
• Separate microservice (Python/ML framework like TensorFlow/PyTorch)
• Receives image, analyzes:
- Color palette extraction
- Style classification (casual, formal, party, traditional)
- Occasion detection (wedding, office, party, etc.)
• Returns recommended product IDs with confidence scores (0-1)
• Backend queries catalog for recommended products
• Returns enriched product data to frontend **Database Considerations:**
• Add aiRecommendations table to track recommendations
• Store image metadata, analysis results, user preferences
• Track recommendation performance for ML model improvement
• A/B testing support for different recommendation algorithms **Future Schema Addition:**

```
model AIRecommendation {
id String @id @default(cuid())
userId String?
sessionId String // For guest users
imageUrl String
imageHash String // For duplicate detection
analysis Json // Store ML model output
productIds String[] // Array of recommended product IDs
scores Float[] // Confidence scores for each product
createdAt DateTime @default(now())

@@index([userId])
@@index([sessionId])
@@map("ai_recommendations")
}
```

Example API Response:

```
POST /api/ai/recommend
Request: FormData with image file
Response:
{
"recommendations": [
{
"productId": "prod_123",
"confidence": 0.92,
"reason": "Color match: Gold matches dress accents"
},
{
"productId": "prod_456",
"confidence": 0.87,
"reason": "Style match: Elegant matches formal occasion"
}
],
"analysis": {
"primaryColors": ["#FFD700", "#FFFFFF"],
"style": "formal",
"occasion": "wedding"
}
}
```

14. Future Extensions

Scalability Enhancements:
• **Redis Caching:** Cache product listings, user sessions, frequently accessed data
• **Elasticsearch:** Advanced full-text search with filters (price range, category, tags)
• **Microservices:** Split into independent services (catalog, order, payment, notification)
• **CDN Integration:** CloudFront/Cloudflare for image delivery and static assets
• **Queue System:** Bull/Redis for async order processing, email sending, inventory updates
• **Database Replication:** Read replicas for scaling read operations
• **Load Balancing:** Distribute traffic across multiple backend instances **Feature Extensions:**
• **User Reviews & Ratings:** Product reviews, ratings, photos from customers
• **Wishlist:** Save favorite products for later
• **Order Tracking:** Real-time shipment tracking with carrier integration
• **Notifications:** Email/SMS for order updates, promotions, stock alerts
• **Admin Dashboard:** Comprehensive admin panel for inventory, orders, analytics
• **Analytics & Reporting:** Sales reports, product performance, user behavior
• **Multi-language:** Internationalization (i18n) support
• **Multi-currency:** Support multiple currencies with automatic conversion
• **Loyalty Program:** Points, rewards, referral system
• **Subscription:** Recurring orders for regular customers **AI/ML Enhancements:**
• **Personalized Recommendations:** User behavior-based product suggestions
• **Image Search:** Upload image to find similar products
• **Chatbot:** AI-powered customer support chatbot
• **Price Optimization:** Dynamic pricing based on demand, inventory, competition
• **Fraud Detection:** ML models to detect fraudulent transactions
• **Inventory Forecasting:** Predict demand and optimize stock levels
• **Search Ranking:** ML-powered search result ranking

15. Example Scenarios

Scenario 1: User Browsing Products
1. User visits homepage: GET /api/catalog/products?page=1&limit;=20
2. Backend queries database:
SELECT * FROM products
WHERE isActive = true AND stock > 0
ORDER BY stock DESC, displayRank ASC, createdAt DESC
LIMIT 20 OFFSET 0
3. For each product, fetch from Sanity CMS:
GET /v1/data/query?query=*[_id=="sanity_prod_123"]
4. Merge database data (price, stock, variants) with Sanity content (images, description, badge)
5. Return 20 products with pagination metadata
6. Frontend displays product cards with images, prices, badges, "Add to Cart" buttons

Scenario 2: Adding to Cart with Discount
1. User selects product "Gold Necklace" with variant "Rose Gold"
2. Backend calculates price:
• Base product price: ■1,299
• Variant "Rose Gold" price: ■1,399 (overrides base)
• Variant discount: 10%
• Final price: ■1,399 * 0.90 = ■1,259.10
3. User adds to cart: POST /api/cart
{ "productId": "prod_123", "variantId": "var_2", "quantity": 2 }
4. Backend creates/updates cart item:
INSERT INTO cart_items (userId, productId, variantId, quantity)
VALUES ('user_123', 'prod_123', 'var_2', 2)
5. Cart total: ■2,518.20 (2 × ■1,259.10)
6. Frontend displays:
- Original price: ■2,798 (2 × ■1,399)
- Discount: 10% (-■279.80)
- Final price: ■2,518.20

Scenario 3: Complete Order Placement & Payment
1. User proceeds to checkout: POST /api/orders
{
"cartItems": [...],
"shippingInfo": { "name": "...", "address": "...", ... }
}
2. Backend creates order:
- Calculates subtotal: ■2,518.20
- Adds shipping: ■50.00
- Total: ■2,568.20
- Status: PENDING
- INSERT INTO orders (userId, totalAmount, status, ...)
3. Backend creates Razorpay order:
POST https://api.razorpay.com/v1/orders
{ "amount": 256820, "currency": "INR", "receipt": "order_xyz" }
4. Razorpay returns: { "id": "order_rzp_123", "status": "created" }
5. Backend stores razorpayId: UPDATE payments SET razorpayId = 'order_rzp_123'
6. Frontend receives: { "razorpayOrderId": "order_rzp_123", "key": "rzp_test_..." }
7. Frontend redirects to Razorpay payment page
8. User pays ■2,568.20 via UPI/Card/Net Banking
9. Razorpay webhook: POST /api/payments/webhook
{

"razorpay_order_id": "order_rzp_123",
"razorpay_payment_id": "pay_xyz",
"razorpay_signature": "..."
}
10. Backend verifies signature, updates:
- Order status: PENDING → CONFIRMED
- Payment status: PENDING → SUCCESS
- Payment razorpayId: 'pay_xyz'
- Reduce inventory for ordered items
11. User receives order confirmation email
12. Frontend shows success page with order details

16. Conclusion

This architecture provides a solid foundation for an artificial jewellery e-commerce platform with the following key strengths: **Scalability:** Modular NestJS backend allows independent scaling of components. Database indexing and caching strategies support high traffic. The architecture can easily scale horizontally. **Maintainability:** Clear separation of concerns, type-safe Prisma ORM, and well-organized modules make the codebase maintainable. Sanity CMS enables non-technical staff to manage content. **Flexibility:** Sanity CMS enables content updates without code deployment. Discount and ordering logic can be easily modified. The system supports various product variants and pricing strategies. **Future-Ready:** Architecture accommodates AI integration, additional payment gateways, and feature extensions without major refactoring. The modular design allows incremental enhancements. **Security:** Payment webhook verification, user authentication, input validation, and secure API design ensure the platform is secure and reliable. **Performance:** Database indexing, efficient queries, content CDN, and caching strategies ensure fast response times and good user experience. **Next Steps for Implementation:**
1. Set up development environment (NestJS, Next.js, PostgreSQL, Sanity)
2. Implement core modules (Catalog, Cart, Order, Payment, User)
3. Create Sanity CMS schemas and initial content
4. Set up Razorpay integration with webhook handling
5. Build frontend components (product listing, cart, checkout, order tracking)
6. Implement authentication and authorization
7. Add error handling, logging, and monitoring
8. Write unit and integration tests
9. Set up CI/CD pipeline
10. Deploy to staging and production environments
11. Future: Integrate AI recommendation module
12. Future: Add analytics and reporting dashboard **Document Version:** 2.0 - Enhanced Edition
Last Updated: December 2024